Research Database

Logan Pollack, Keely, Bomba, Sofia Mancini, Sean

・CSC 436・Spring 2026・

Application Overview (Logan)

Our database application is a research database for university researchers. Its primary purpose is to provide an easy way to view tasks for each experiment being run by a research team. The database also provides an easy way to view each department in the university and also who is in each research group. The main feature of the application is to provide a simple and easy way to track tasks and experiments for a research project.

Database Design (Keely)

Explain the structure of your database using your E-R Diagram. It should focus on:

- *Major entities and attributes*
- *Primary keys*
- *Relationships*
- *Important constraints*

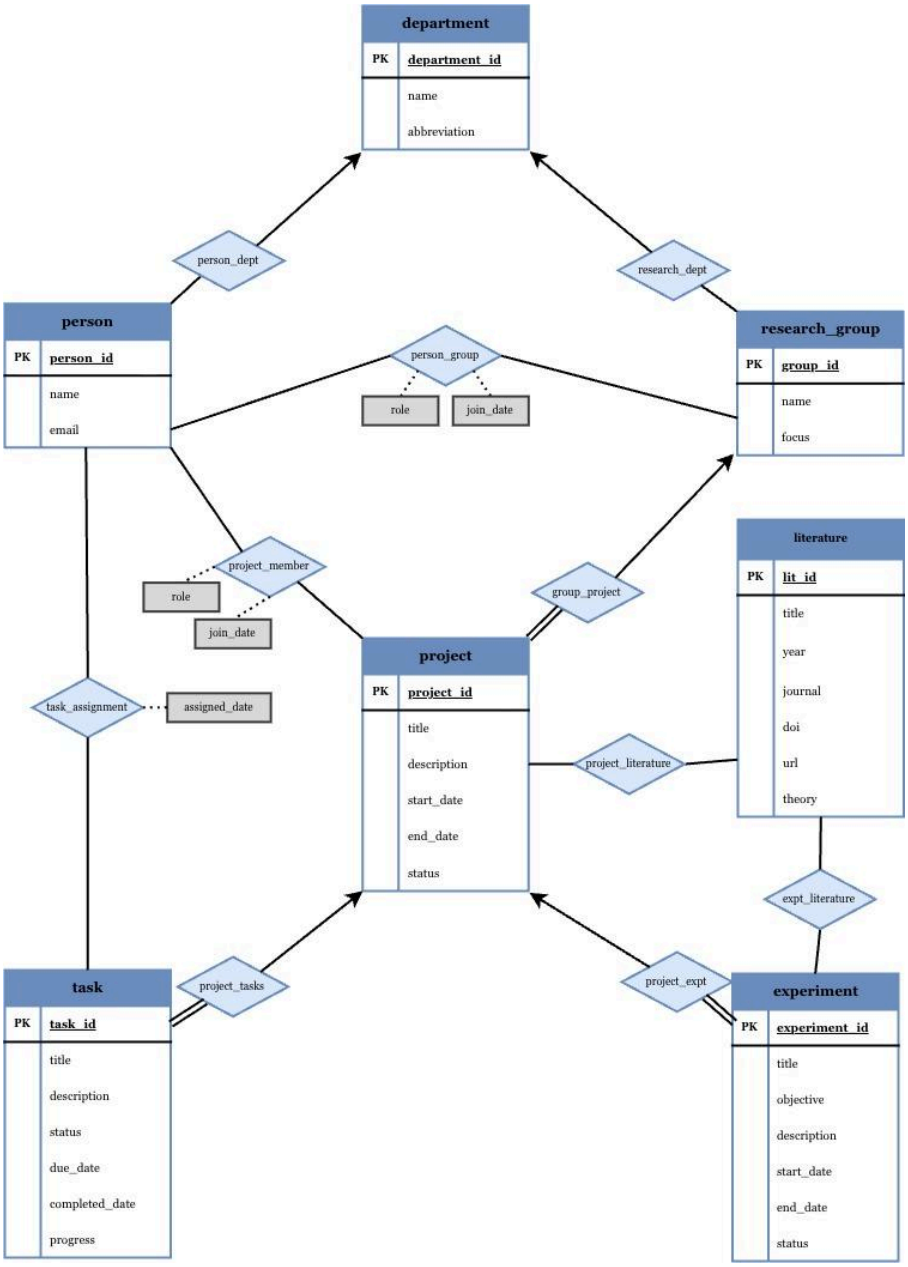
The database models a research environment with core entities: person, department, research_group, project, task, experiment, and literature, each identified by a single primary key (person_id, department_id, group_id, project_id, task_id, experiment_id, lit_id). Each table stores only attributes directly related to that entity, such as person (name, email), project (title, description, dates, status), and task (status, progress, due dates), as shown in the schema and implementation documentation (pictured in the normalization section). Relationships between entities are implemented using junction tables with foreign keys, including person_group, project_member, task_assignment, project_expt, project_literature, expt_literature, research_dept, group_project, and person_dept, allowing the database to represent many-to-many relationships without duplicating data.

Include figure(s) of your E-R diagram and explain how your design supports the functionality of your application.

Our ER diagram centers on the project entity, which connects tasks, experiments, literature, and research groups, which supports the application's core functionality of tracking research work across multiple dimensions. Relationships such as project_tasks (1:M ~ each task belongs to a project) and task_assignment (M:N ~ tasks can be assigned to multiple people w/ an assigned_date attribute) are enforced through foreign keys, ensuring that all references correspond to existing parent records.

Membership relationships such as person_group and project_member store role and join_date on the relationship itself, ensuring that role depends on the specific context rather than the person entity. Referential integrity is enforced through foreign keys in all junction tables, and constraints such as valid status values, date ordering, and required fields ensure consistent and valid data across the system.This structure supports flexible participation, consistent role tracking, and reliable linking of data across all application features.

E-R Diagram



E-R diagram of the research database showing core entities and relationships, with all many-to-many (M:N) relationships resolved through junction tables (e.g., project_member, person_group). Relationship attributes (role, join_date) are stored on these junctions, reflecting that they depend on the full composite relationship rather than a single entity.

Database Implementation (Logan)

Our Database was implemented in SQL and split into many tables. We populated our database by asking an AI model to generate dummy data for us. The first table is the department table. This contains the department_id, name, and its abbreviation. The primary key in this table is department_id.

```
DROP TABLE IF EXISTS `department`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `department` (
  `department_id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  `abbreviation` varchar(20) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  PRIMARY KEY (`department_id`)
) ENGINE=InnoDB AUTO_INCREMENT=31 DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_unicode_ci;
```

This table is primarily related to the project table as each project is associated with a single department.

The second table we have is our experiment table. This table has a title, the status of the experiment and the start and end date. It has an important constraint in forcing the start date to be before the end data. It also has the constraint of having the status be Active, Completed, Cancelled or On Hold with a primary key of experiment_id. It has a strong relationship with the literature table and project table.

```
DROP TABLE IF EXISTS `experiment`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `experiment` (
  `experiment_id` int NOT NULL AUTO_INCREMENT,
  `title` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  `objective` text CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci,
  `description` text CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci,
  `start_date` date DEFAULT NULL,
  `end_date` date DEFAULT NULL,
  `status` varchar(50) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  PRIMARY KEY (`experiment_id`),
  CONSTRAINT `chk_experiment_dates` CHECK (((`end_date` is null) or (`start_date` is null) or (`end_date` > `start_date`))),
  CONSTRAINT `chk_experiment_status` CHECK ((`status` in (_utf8mb3'Active',_utf8mb3'Completed',_utf8mb3'Cancelled',_utf8mb3'On Hold'))),
  CONSTRAINT `chk_experiment_title` CHECK ((trim(`title`) <> _utf8mb4''))
) ENGINE=InnoDB AUTO_INCREMENT=32 DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

The third table we have is the project table, which contains the project's title, status, start date and end date, description, and project_id. Its primary key is the project_id and has a strong relationship with experiments and literature. It is also related to the person table as it has a sub table named project_member which stores the people working on the specified project. It has several integrity constraints, the strongest being that its status is only able to be Active, Completed, Cancelled, or On Hold.

```
DROP TABLE IF EXISTS `project`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `project` (
  `project_id` int NOT NULL AUTO_INCREMENT,
  `title` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  `description` text CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci,
  `start_date` date DEFAULT NULL,
  `end_date` date DEFAULT NULL,
  `status` varchar(50) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  PRIMARY KEY (`project_id`),
  CONSTRAINT `chk_project_dates` CHECK (((`end_date` is null) or (`start_date` is null) or (`end_date` > `start_date`))),
  CONSTRAINT `chk_project_status` CHECK ((`status` in (_utf8mb3'Active',_utf8mb3'Completed',_utf8mb3'Cancelled',_utf8mb3'On Hold'))),
  CONSTRAINT `chk_project_title` CHECK ((trim(`title`) <> _utf8mb4''))
) ENGINE=InnoDB AUTO_INCREMENT=32 DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

Our Fourth Table is the Person table which holds the name, email, password, ID, and role. Its primary key is the person's ID which is unique. It has several constraints including forcing the email to be valid and the email to not be empty. The table's relationships are with projects, departments, and groups.

```
DROP TABLE IF EXISTS `Person`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `Person` (
  `ID` int NOT NULL AUTO_INCREMENT,
  `name` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  `email` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  `password` varchar(255) COLLATE utf8mb3_unicode_ci NOT NULL,
  `role` varchar(100) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  PRIMARY KEY (`ID`),
  CONSTRAINT `chk_person_email` CHECK ((`email` like _utf8mb3'%@%.%')),
  CONSTRAINT `chk_person_email_notempty` CHECK ((trim(`email`) <> _utf8mb3'')),
  CONSTRAINT `chk_person_name` CHECK ((trim(`name`) <> _utf8mb3''))
) ENGINE=InnoDB AUTO_INCREMENT=32 DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

The fifth table we have is the literature table. This table contains the literature's title, year it was published and the journal it originated from. It also contains its url and main abstract/theory. It has a constraint forcing the year to be published between 1900 and 2100, and also makes sure the url is valid. Another constraint that is included is making sure a title for the literature exists. Literature is only related to the experiment table and is not as closely related to projects.

```
DROP TABLE IF EXISTS `literature`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `literature` (
  `lit_id` int NOT NULL AUTO_INCREMENT,
  `title` varchar(255) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci NOT NULL,
  `year` year NOT NULL,
  `journal` varchar(255) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci DEFAULT NULL,
  `doi` varchar(255) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci DEFAULT NULL,
  `url` varchar(255) CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci DEFAULT NULL,
  `theory` text CHARACTER SET utf8mb3 COLLATE utf8mb3_unicode_ci,
  PRIMARY KEY (`lit_id`),
  CONSTRAINT `chk_literature_doi` CHECK (((`doi` is null) or (`doi` like _utf8mb3'10.%'))),
  CONSTRAINT `chk_literature_title` CHECK ((trim(`title`) <> _utf8mb3'')),
  CONSTRAINT `chk_literature_url` CHECK (((`url` is null) or (`url` like _utf8mb4'http%'))),
  CONSTRAINT `chk_literature_year` CHECK ((`year` between 1900 and 2100))
) ENGINE=InnoDB AUTO_INCREMENT=31 DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

Finally we have our task table. This table contains the tasks title, status, progress and its due date with a primary key of task_id. It has similar constraints to the project table where status can only be either Completed, In Progress, Not Started or Cancelled. Its progress must also be between 0 and 100 and it must have a valid title. Tasks are closely related to projects, having their own relationship table called project_tasks as well.

```
DROP TABLE IF EXISTS `project_tasks`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `project_tasks` (
  `project_id` int NOT NULL,
  `task_id` int NOT NULL,
  PRIMARY KEY (`project_id`,`task_id`),
  KEY `task_id` (`task_id`),
  CONSTRAINT `project_tasks_ibfk_1` FOREIGN KEY (`project_id`) REFERENCES `project` (`project_id`),
  CONSTRAINT `project_tasks_ibfk_2` FOREIGN KEY (`task_id`) REFERENCES `task` (`task_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3 COLLATE=utf8mb3_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

Normalization (Keely)

Explain how your database design satisfies Third Normal Form (3NF). It should focus on:

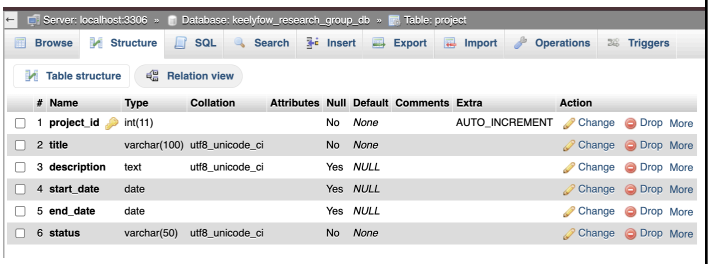
- *Reducing redundancy*
- *Organizing data into appropriate tables*
- *Key design decisions that improved consistency*

Include examples or figure(s) where helpful.

The database satisfies Third Normal Form (3NF) by organizing data into separate tables where each non-key attribute depends only on its primary key, with no partial or transitive dependencies. Each entity table stores only its own attributes, while many-to-many relationships such as person_group, project_member, task_assignment, and project_literature are separated into junction tables, eliminating repeating groups and preventing redundancy across the schema.

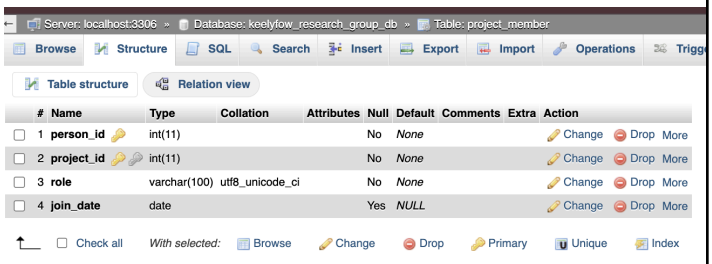
A key design decision was removing multi-valued attributes and placing relationship-specific data in junction tables, such as storing role and join_date in person_group and project_member instead of the person table. In earlier work, the literature.authors attribute stored comma-separated values, which violated First Normal Form; this was identified and removed, eliminating a source of redundancy and inconsistency even though the separate author table was not included in the final schema. Overall, the structure ensures that data is stored once, relationships are handled explicitly, and updates do not introduce inconsistencies. In junction tables, attributes such as role and join_date depend on the full composite key rather than part of it, satisfying both 2NF and 3NF.

Project Table Structure



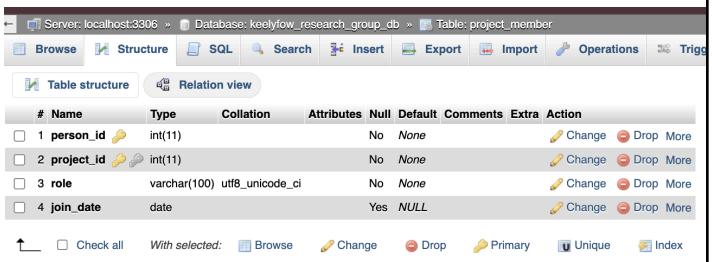
Structure of the project table showing project_id as the primary key and attributes (title, description, dates, status) that depend only on that key, illustrating that project data is stored independently with no redundancy or transitive dependencies (3NF).

Project_Member Table Structure



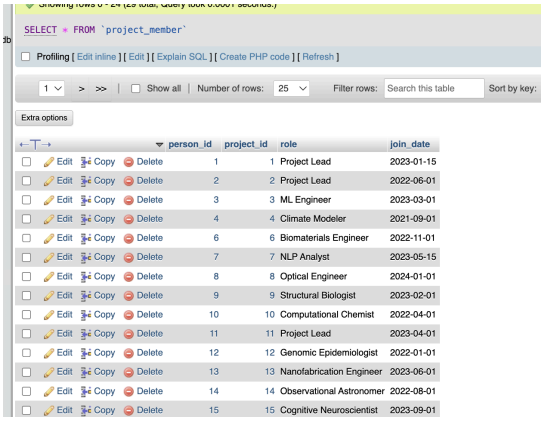
Structure of the project_member junction table implementing a (M:N) relationship between person and project using foreign keys (person_id, project_id), with role and join_date stored as relationship attributes that depend on the full composite key.

Project_Member Table Structure



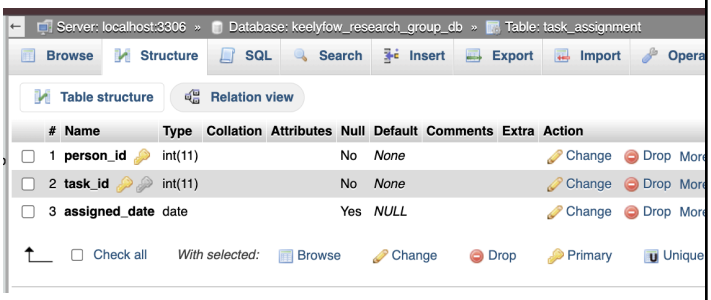
Structure of the project_member junction table implementing a (M:N) relationship between person and project using foreign keys (person_id, project_id), with role and join_date stored as

Project_Member Sample Data



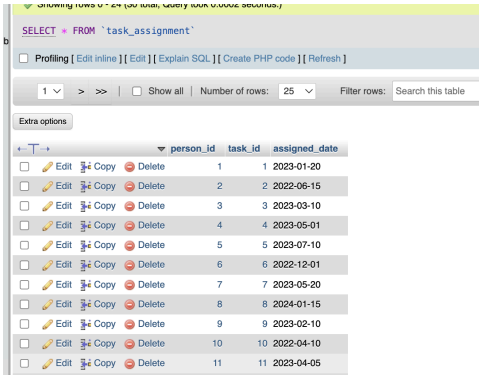
Sample data from project_member showing person_id and project_id as foreign key references rather than duplicating entity attributes (e.g., names or titles), with role and join_date varying per relationship, demonstrating normalized storage and dependency on the composite key.

Task_Assignment Table Structure



Structure of the task_assignment junction table representing a (M:N) relationship between person and task, with assigned_date stored as a relationship attribute dependent on the specific person–task pairing.

Task_Assignment Sample Data



Sample data from task_assignment showing multiple assignments across person_id and task_id pairs, with assigned_date varying per relationship and no duplication of task or person attributes, reinforcing use of foreign keys and normalized design.

Application Functionality & Database Integration (Sofia)

Query 1: Multi-Table Join for Research Group Member Lookup

This query retrieves all members belonging to a specific research group, along with their contact information, role, and join date. It demonstrates a multi-table JOIN across three related tables and is one of the most frequently executed queries in the application, powering the Members page and the member_contact_view database view.

```
mysql> SELECT p.name, p.email, pg.role, pg.join_date, rg.name AS group_name
    -> FROM Person p
    -> JOIN person_group pg ON p.ID = pg.person_id
    -> JOIN research_group rg ON pg.group_id = rg.group_id
    -> WHERE rg.name = 'Genomics Lab'
    -> ORDER BY pg.join_date ASC;
+---------------+-----------------------+---------------------+------------+---------------+
| name          | email                 | role                | join_date  | group_name    |
+---------------+-----------------------+---------------------+------------+---------------+
| Alice Johnson | alice.johnson@uni.edu | Principal Investigator | 2020-01-10 | Genomics Lab |
+---------------+-----------------------+---------------------+------------+---------------+
1 row in set (0.016 sec)
```

Results:

As shown in the figure above, the query returns one row for the Genomics Lab, identifying Alice Johnson as the Principal Investigator who joined on January 10, 2020. The result set includes her email address, role, join date, and the group name resolved from the research_group table. The query executed in 0.016 seconds against a dataset of 30 members across 30 research groups.

How the Query Supports the Application

This query reflects the relational structure at the core of the database design. Rather than storing group membership and contact details in a single flat table, the schema separates concerns across Person, person_group, and research_group, connected through foreign key relationships. The JOIN chain traverses these relationships to assemble a complete picture of group membership from normalized data.

In the application, this query is used in two ways. On the members.php page it populates the member table for all logged-in users. It is also the basis of the member_contact_view database view, which restricts email visibility to authenticated users only, implementing a basic data privacy layer at the database level. The WHERE clause accepts a dynamic group name parameter, and in the PHP implementation the value is passed as a bound prepared statement parameter to prevent SQL injection.

Query 2: Aggregate Count with Multi-Table Join for Department Project Summary

This query calculates the total number of projects associated with each department by traversing the relationship chain connecting departments to research groups to projects. It demonstrates the use of aggregate functions, group by, left join, and count(distinct) in combination, and powers the Projects column on the Departments page of the application.

```
mysql> SELECT d.name AS department, d.abbreviation,
    ->        COUNT(DISTINCT gp.project_id) AS project_count
    -> FROM department d
    -> LEFT JOIN research_dept rd ON d.department_id = rd.department_id
    -> LEFT JOIN group_project gp ON rd.group_id = gp.group_id
    -> GROUP BY d.department_id, d.name, d.abbreviation
    -> ORDER BY project_count DESC
    -> LIMIT 10;
+------------------------+--------------+---------------+
| department             | abbreviation | project_count |
+------------------------+--------------+---------------+
| Biology                | BIO          |             3 |
| Biochemistry           | BIOC         |             3 |
| Chemistry              | CHEM         |             2 |
| Physics                | PHYS         |             2 |
| Statistics             | STAT         |             2 |
| Environmental Science  | ENV          |             2 |
| Biomedical Engineering | BME          |             2 |
| Psychology             | PSY          |             2 |
| Data Science           | DS           |             2 |
| Computer Science       | CS           |             1 |
+------------------------+--------------+---------------+
10 rows in set (0.043 sec)
```

Results:

As shown in the figure above, the query returns the top 10 most active departments ordered by project count. Biology and Biochemistry lead with 3 projects each, followed by Chemistry, Physics, Statistics, Environmental Science, Biomedical Engineering, Psychology, and Data Science each contributing 2 projects, and Computer Science with 1. The query executed in 0.043 seconds, slightly longer than Query 1 due to the additional aggregation and grouping operations across all 30 departments.

How the Query Supports the Application

This query demonstrates how the normalized schema handles many-to-many relationships across multiple entities. Departments do not link directly to projects and instead the relationship is bridged through research_dept and group_project, reflecting the real-world structure where a department belongs to a research group which in turn owns projects. The left join is used rather than an inner join so that departments with zero projects still appear in the results rather than being silently excluded.

count(distinct gp.project_id) is used rather than a plain count(*) to avoid overcounting in cases where a department is linked to multiple research groups that share the same project. In the application this query populates the Departments page table, and each project count value renders as a clickable link that passes ?dept_id= to projects.php, dynamically

filtering the projects list to show only those associated with the selected department. This creates a natural drill-down navigation pattern between the two pages entirely driven by the relational structure of the database.

Query 3: Update with Verification Select for Experiment Record Modification

This query modifies an existing experiment record, updating its title, status, and end date in a single operation. It represents the core data modification pattern used throughout the application wherever a user edits a record, and is paired with a verification SELECT to confirm the change was applied correctly. This query type is the most security-sensitive in the application as it directly mutates stored data.

```
mysql> UPDATE experiment
    ->
    -> SET title      = 'CRISPR Transfection Trial 1 ( Revised)',
    ->
    ->     status     = 'Completed',
    ->
    ->     end_date   = '2023-04-30'
    ->
    -> WHERE experiment_id = 1;
Query OK, 1 row affected (0.078 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT experiment_id, title, status, end_date
    -> FROM experiment
    -> WHERE experiment_id = 1;
+---------------+-----------------------------------------+-----------+------------+
| experiment_id | title                                   | status    | end_date   |
+---------------+-----------------------------------------+-----------+------------+
|             1 | CRISPR Transfection Trial 1 ( Revised) | Completed | 2023-04-30 |
+---------------+-----------------------------------------+-----------+------------+
1 row in set (0.005 sec)
```

Results:

As shown in the figure above, MySQL confirms Query OK, 1 row affected with Rows matched: 1 Changed: 1 Warnings: 0, indicating the update was applied cleanly to exactly one record with no issues. The verification Select immediately following confirms the updated values are correctly stored and the title now reads CRISPR Transfection Trial 1 (Revised), status shows Completed, and the end date is recorded as 2023-04-30.

How the Query Supports the Application

This query represents the pattern behind every edit page in the application, experiment_edit.php, project_edit.php, task_edit.php, literature_edit.php, and member_edit.php all use the same update structure. The where experiment_id = 1 clause is critical for data integrity, ensuring only the intended row is modified. Without it the update would affect every row in the table.

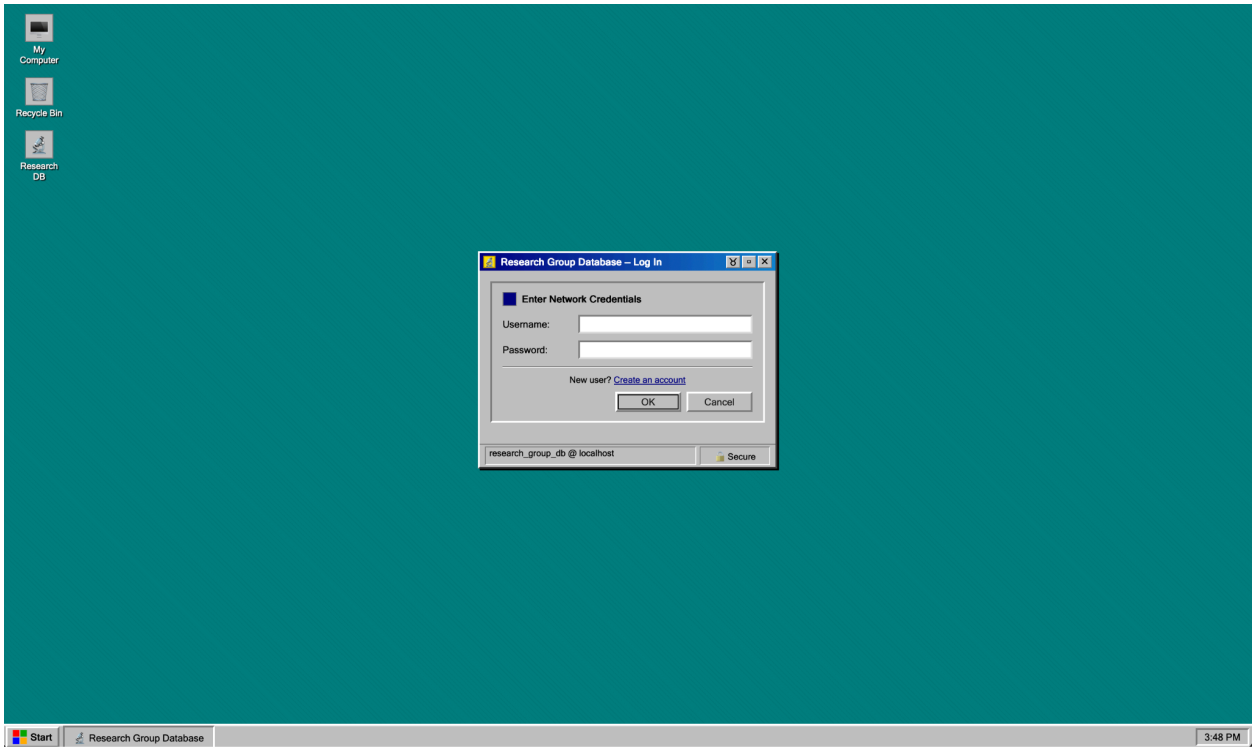
In the PHP application this query never runs with raw user input. Instead it is executed as a prepared statement through the pdo() helper function, where user-supplied values are passed as bound parameters:

```php
pdo($pdo,
    "UPDATE experiment SET title=:title, objective=:objective,
     description=:description, start_date=:start_date,
     end_date=:end_date, status=:status
     WHERE experiment_id=:id",
    [
        'title'       => $title,
        'objective'   => $objective,      You, 2 weeks ago • up
        'description' => $description,
        'start_date'  => $start_date,
        'end_date'    => $end_date,
        'status'      => $status,
        'id'          => $id,
    ]
);
```

The named placeholders :title, :status, :end_date, and :id are resolved by PDO at execution time, completely separating the query structure from the data values. This prevents SQL injection regardless of what a user submits through the form. Server-side validation also runs before the query executes, enforcing constraints such as valid status values and correct date ordering, providing a second layer of data integrity on top of the database-level CHECK constraints already defined on the experiment table.

Application Interface (GUI)

Application Interface — Login Page



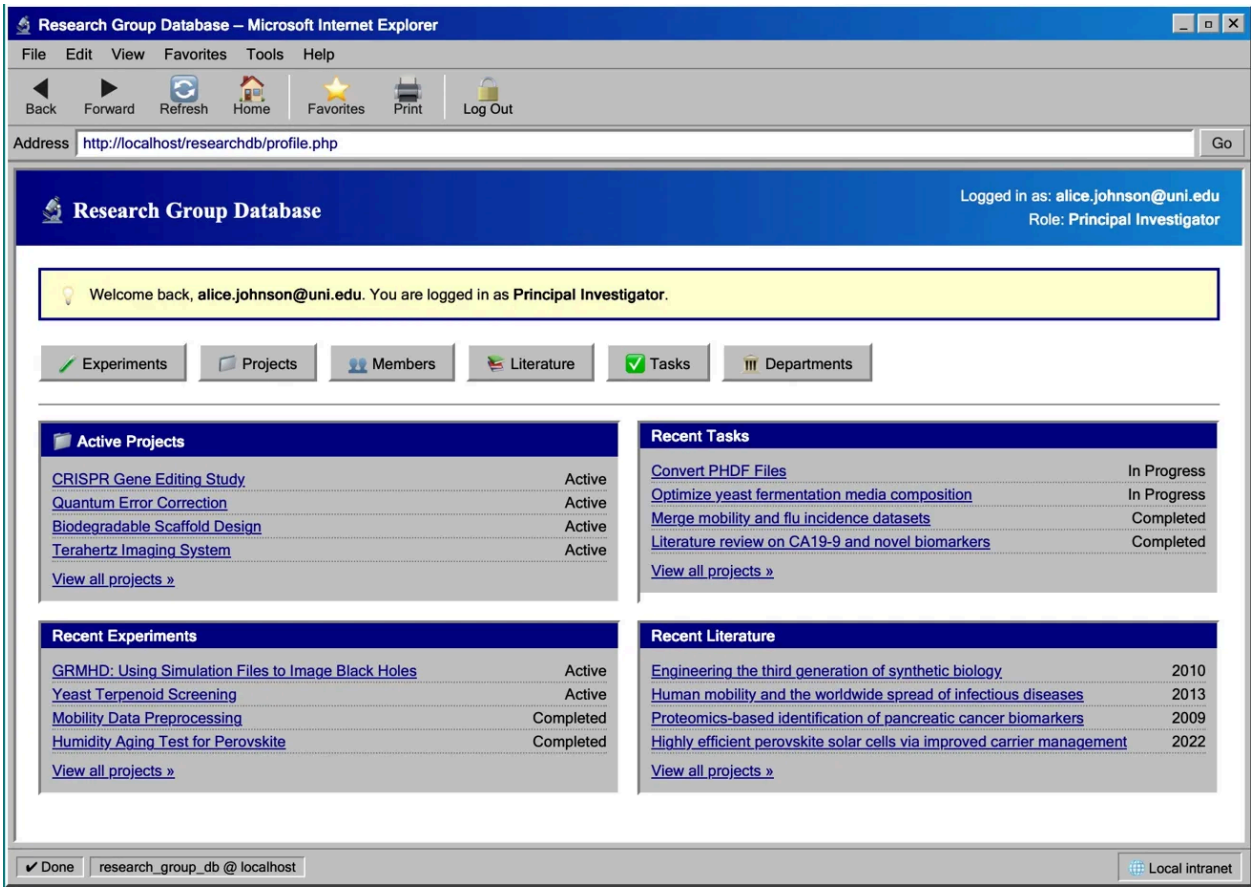
The login page is designed as a Windows 98 style dialog window, rendered against the characteristic teal desktop background of the era. The aesthetic is applied consistently throughout the entire application, giving the interface a cohesive retro identity that distinguishes it from conventional web applications.

The layout is intentionally minimal, just a single centered window prompts the user for a username and password within a labeled credentials panel. Two buttons, OK and Cancel, handle form submission and reset respectively. A "New user? Create an account" link below the divider provides access to the self-registration page without requiring any additional navigation.

New users who register through the Create an account flow are assigned a Viewer role by default, giving them read-only access to the database until a Principal Investigator promotes them to a full member role. This means the login page serves as the gateway to two distinct user flows, returning members logging into their existing accounts, and new users joining the system for the first time.

The status bar at the bottom displays the active database connection and a secure indicator, providing users with basic connection context before they authenticate.

Application Interface — Landing Page



After logging in, users are brought to the dashboard, rendered as a full Internet Explorer window complete with a menu bar, navigation toolbar, and address bar showing the current page URL. The logged-in user's email and role are displayed in the top right corner of the page header, and a welcome banner confirms their session and access level.

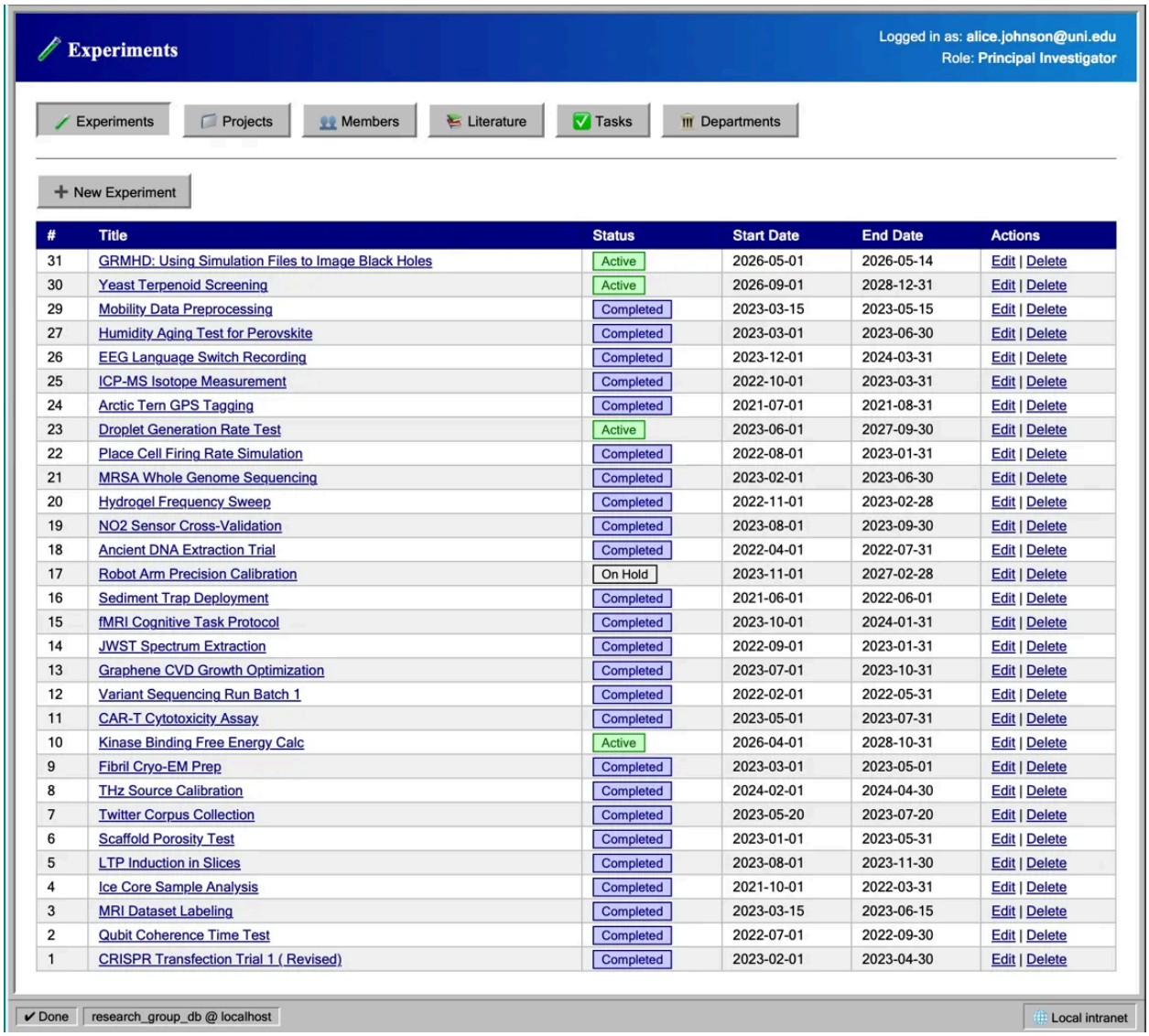
The main navigation is a row of six quick-link buttons: Experiments, Projects, Members, Literature, Tasks, and Departments, all of which persist across every page in the application, allowing users to move between sections without returning to the dashboard first.

Below the navigation the dashboard is divided into four summary panels arranged in a two-column grid, giving the user an at-a-glance overview of their research projects. Active Projects and Recent Experiments are shown on the left, while

Recent Tasks and Recent Literature appear on the right. Each panel lists the four most recent or active records with their current status, and a "View all" link at the bottom of each panel navigates to the full table for that section.

The toolbar at the top provides Back, Forward, Refresh, and Home navigation as well as a Log Out button, keeping all primary actions accessible from any page. The status bar at the bottom confirms the database connection and displays a Local Intranet indicator, consistent with the IE aesthetic throughout the application.

Application Interface — Information Pages (Experiments Page)



Each section of the application follows the same layout pattern, illustrated here by the Experiments page. The page header displays the section title and the logged-in user's credentials, while the persistent navigation bar below it allows immediate movement to any other section without backtracking.

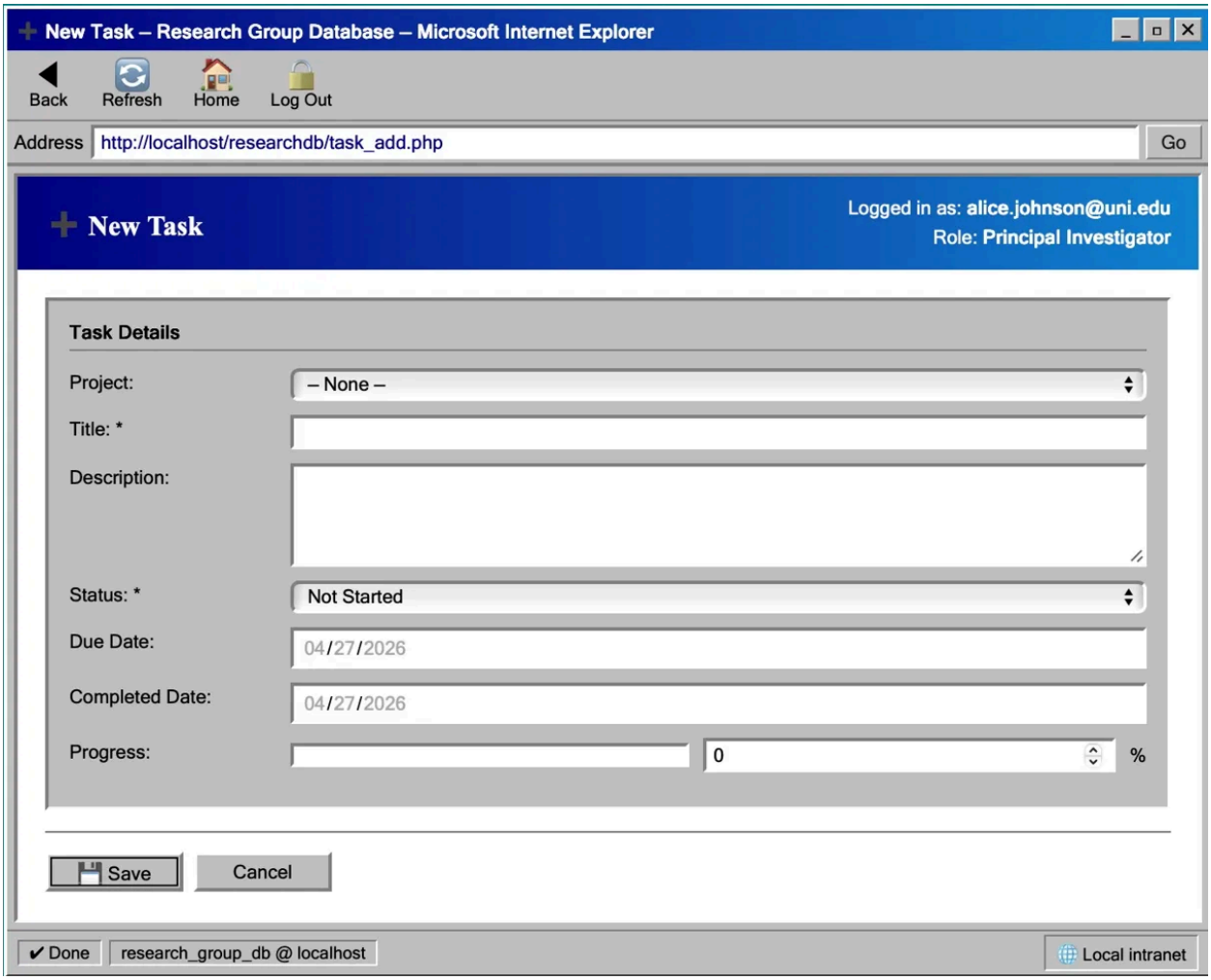
A New Experiment button sits above the data table and is only visible to users with sufficient permissions, in this case the logged-in Principal Investigator, Alice Johnson. Viewer accounts see the same table but without the ability to add, edit, or delete records.

The main content is a full-width data table listing all 31 experiments in the database, ordered with the most recent first. Each row displays the record ID, title as a clickable link, a color-coded status badge, start and end dates, and an Actions column containing Edit and Delete links. The status badges use distinct colors to communicate state at a glance, green for Active, blue for Completed, and neutral for On Hold, these allow users to scan the table quickly without reading each status label individually.

The Actions column reflects the role-based permission system directly in the UI. As a Principal Investigator, Alice sees both Edit and Delete for every row. A Member would see Edit but not Delete, and a Viewer would see the buttons but receive an access denied message on click. This makes the permission model transparent to the user without requiring them to navigate away to discover what they can and cannot do.

The same table structure, navigation bar, permission-aware action buttons, and status badge system are used consistently across the Projects, Literature, Tasks, Members, and Departments pages, ensuring that users who learn to navigate one section can immediately apply that knowledge to all others. The specific information and functionality shifts slightly across pages, for example Tasks include an additional progress bar and the Departments page gives an option to display all projects currently active for that department.

Application Interface — Add/Edit Form (New Task)



Clicking any Add or Edit button opens a dedicated form page, illustrated here by the New Task page. The toolbar simplifies to Back, Refresh, Home, and Log Out since forward navigation is not applicable when creating a new record, and the address bar reflects the current page URL.

The form is contained within a labeled Task Details panel and presents all editable fields for the record in a consistent label-and-input layout. Required fields are marked with an asterisk. The Task form includes a Project dropdown, Title, Description, Status, Due Date, Completed Date, and a Progress field.

A key aspect of the form design is that dropdown fields are populated dynamically from the database at page load rather than being hardcoded. The Project dropdown, for example, queries the project table and lists all available projects, meaning newly added projects automatically appear as options without any changes to the application code. The same principle applies to the Status dropdown, which is constrained to the valid values defined by the Check constraint on the task table, ensuring the UI and the database stay in agreement about what values are acceptable.

Save and Cancel buttons sit below the form. On a successful save the user is redirected back to the Tasks list with a confirmation banner, and on a validation error the form re-renders with the user's input preserved and an error message describing what needs to be corrected.

The same form pattern is used for adding and editing Experiments, Projects, Literature, and Members, with each form's fields reflecting the columns and constraints of its corresponding database table.

Frontend & Backend Integration

Technologies Used

The application is built on a four-layer stack. The frontend is written in HTML and CSS, styled entirely inline to maintain the Windows 98 aesthetic without external stylesheets or frameworks. JavaScript is used sparingly for client-side interactions such as the live clock in the taskbar, the real-time progress bar update on the Task form, and confirmation dialogs before destructive actions. The backend is written in PHP, which handles all session management, authentication, authorization, and database communication. MySQL serves as the database layer, running locally and accessed from the PHP backend through PDO.

User Interactions

All user interaction flows through standard HTML forms that POST data to PHP scripts. When a user navigates to a page such as experiments.php, PHP executes a SELECT query against MySQL through the shared pdo() helper function and renders the results directly into the HTML table. No page refresh or asynchronous request is involved — the data is fetched server-side on every page load, ensuring the interface always reflects the current state of the database.

When a user submits a form such as adding a new experiment, editing a project, or updating a task, the browser sends a POST request to the corresponding PHP handler. The handler validates the input server-side, constructs a prepared statement, and executes it against the database. On success the user is redirected back to the list page with a confirmation banner. On failure the form re-renders with the submitted values preserved and an inline error message describing the issue.

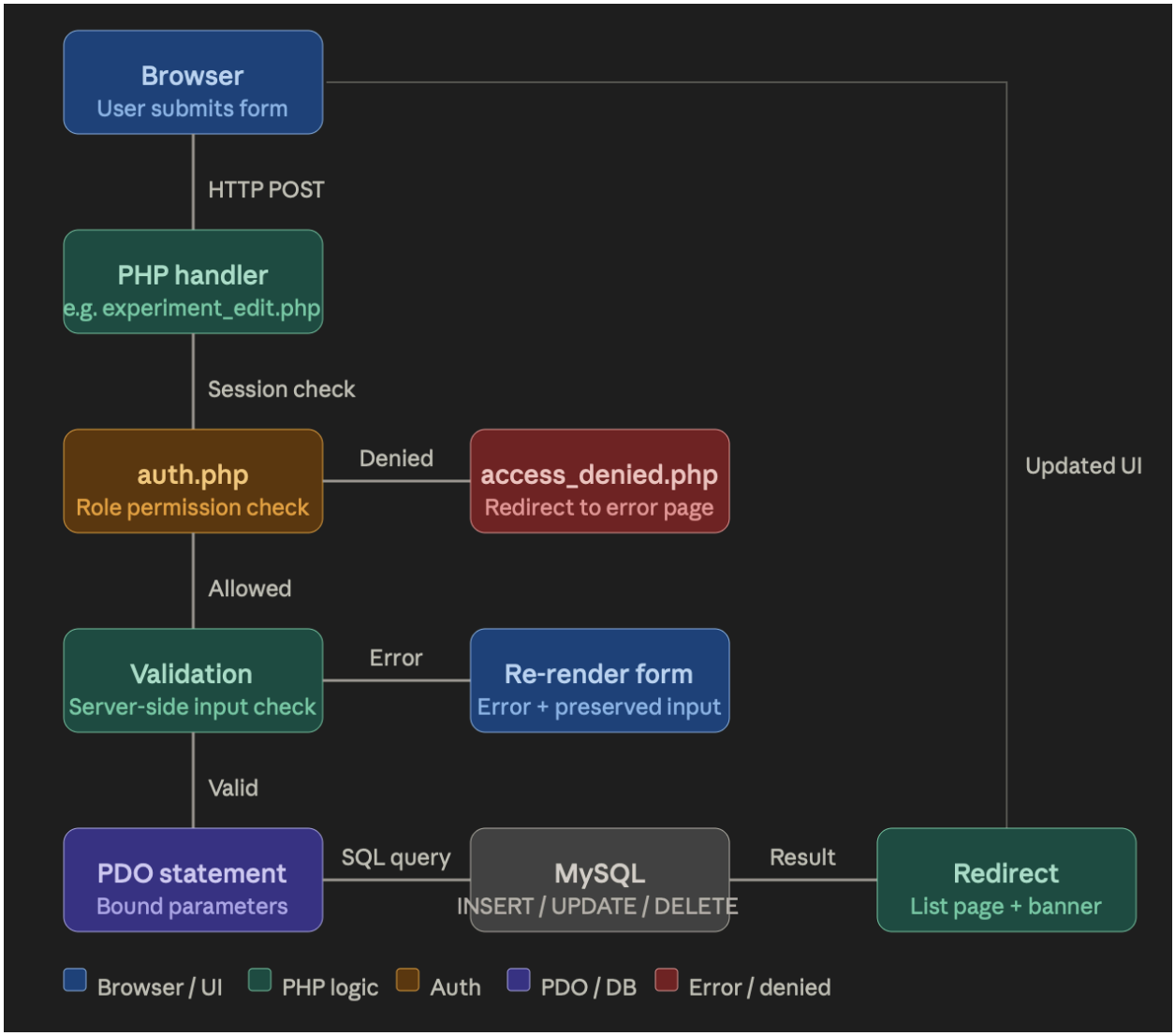
How Interface Actions Affect the Database

Each form action maps directly to a SQL operation:

User Action	PHP File	SQL Operation
Submit New Experiment form	experiment_add.php	**INSERT INTO** experiment
Submit Edit Experiment form	experiment_edit.php	**UPDATE** experiment
Confirm Delete on experiment	experiment_delete.php	**DELETE FROM** expt_literature, **DELETE FROM** project_expt, **DELETE FROM** experiment
Submit New Member form	member_add.php	**INSERT INTO** Person
Submit Assign Role form	member_add.php	**UPDATE** Person **SET** role
Register new account	register.php	**INSERT INTO** Person with role Viewer

Delete operations are the most involved since before deleting a parent record the handler first removes all related rows from junction tables to avoid foreign key constraint violations. For example, deleting a project triggers cascading deletes across project_literature, project_expt, project_tasks, project_member, and group_project before the project row itself is removed.

Flow Chart



Security, Privacy & Data Protection (Sean)

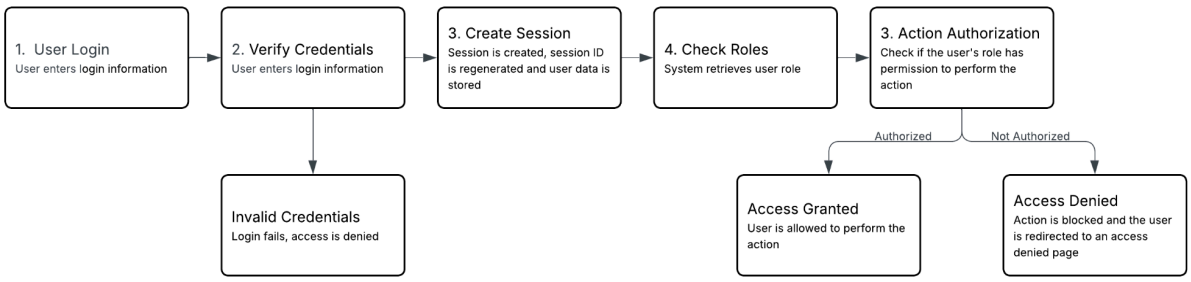
Our application implements several layers of security in order to protect user data, ensure database interactions are safe and secure, and restrict access based on user roles. One of our primary security measures is the use of prepared statements with bound parameters for all queries, ensuring that user input cannot be directly embedded into SQL commands. This allows our application to prevent SQL injection attacks by separating the structure of our queries from user data. For example, if a user attempted to inject an SQL statement into a text field, our database will treat all text as plain text instead of code.

Another security measure we have taken for our database is ensuring that server-side validation is performed prior to any database operation's execution. Input data such as required fields are checked within our PHP handlers to ensure that they meet expected constraints before they are processed, making sure that malicious or invalid data cannot be stored.

Password security is ensured by hashing all user passwords using bcrypt before they are stored within the database. When a user is logging in, passwords are verified using a secure comparison function rather than simply being decrypted, ensuring that user passwords will still remain protected in the event that the database becomes compromised.

A session-based system is used to handle authentication, where users have to log in with valid credentials before they access the application. When the user logs in successfully, session data is then stored and used to maintain the user's authenticated state across different pages. Session IDs are regenerated to reduce the risk of session hijacking.

Authorization is enforced using our role-based access control system, where users are assigned different roles such as viewer, member, or principal investigator. Before allowing actions within the database such as editing records, permissions for the user's role are checked. For example, a viewer would not be allowed to make edits to the database, but they would be allowed to access read-only data. Actions that are not authorized by a user's role are blocked and the user is redirected to an access denied page.

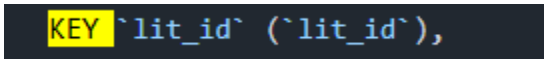


Optimization & Reliability Enhancements (Bamba)

Explain how your group improved performance and reliability. It should:

- Identify indexed columns and explain their impact on performance.
- Identify one important query and explain how it was analyzed, improved, or optimized for efficiency.
- Identify one multi-step database operation and explain how transactions preserve data integrity and consistency.

Include figure(s) showing indexed columns, the optimized query and analysis, and the multi-step database operation.



To keep the application running smoothly, certain columns that are used frequently when connecting tables are indexed. For example, lit_id in expt_literature, project_id in group_project, and task_id in project_tasks all have indexes. Without them, the database would have to scan every row to find matches, which gets slow pretty quickly. With indexing, lookups are much faster, which makes a noticeable difference on pages like Members and Departments that pull data from multiple tables.

```
mysql> SELECT d.name AS department, d.abbreviation,
    ->        COUNT(DISTINCT gp.project_id) AS project_count
    -> FROM department d
    -> LEFT JOIN research_dept rd ON d.department_id = rd.department_id
    -> LEFT JOIN group_project gp ON rd.group_id = gp.group_id
    -> GROUP BY d.department_id, d.name, d.abbreviation
    -> ORDER BY project_count DESC
    -> LIMIT 10;
+------------------------+--------------+---------------+
| department             | abbreviation | project_count |
+------------------------+--------------+---------------+
| Biology                | BIO          |             3 |
| Biochemistry           | BIOC         |             3 |
| Chemistry              | CHEM         |             2 |
| Physics                | PHYS         |             2 |
| Statistics             | STAT         |             2 |
| Environmental Science  | ENV          |             2 |
| Biomedical Engineering | BME          |             2 |
| Psychology             | PSY          |             2 |
| Data Science           | DS           |             2 |
| Computer Science       | CS           |             1 |
+------------------------+--------------+---------------+
10 rows in set (0.043 sec)
```

One query worth noticing is the one that shows how many projects each department has on the Departments page. A simple COUNT could have been used, but it would give incorrect results when multiple research groups are linked to the same project. To avoid double counting, COUNT(DISTINCT) is used instead. A LEFT JOIN is also used instead of an INNER JOIN so that departments with no projects still appear on the page rather than being excluded entirely.

```php
// Handle confirmed deletion
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['confirm_delete'])) {
    // Remove junction table rows first to avoid FK constraint errors
    pdo($pdo, "DELETE FROM project_literature WHERE project_id = :id", ['id' => $id]);
    pdo($pdo, "DELETE FROM project_expt WHERE project_id = :id",       ['id' => $id]);
    pdo($pdo, "DELETE FROM project_tasks WHERE project_id = :id",      ['id' => $id]);
    pdo($pdo, "DELETE FROM project_member WHERE project_id = :id",     ['id' => $id]);
    pdo($pdo, "DELETE FROM group_project WHERE project_id = :id",      ['id' => $id]);
    pdo($pdo, "DELETE FROM project WHERE project_id = :id",            ['id' => $id]);
    header('Location: projects.php?deleted=1');
    exit;
}
```

Deleting a project is more than just removing a single row, it requires six separate database operations. Before deleting the project itself, the application first removes related rows in project_literature, project_expt, project_tasks, project_member, and group_project, otherwise the database would raise a foreign key error since those tables still reference the project. Once those are cleared the main project row can be deleted safely. Wrapping this process in a transaction ensures that if any step fails, all changes are rolled back, keeping the database consistent instead of being left partially deleted.

Reflection & Final Improvements

Explain how your group reflected on strengths, challenges, and lessons learned to produce a polished, production-ready database application.

Strengths

Discuss the strengths of your database application. It should address:

- *What strengths were highlighted in feedback*
- *What worked well in your application*

The application performed well across both the database and interface layers. The schema is normalized to 3NF and consistently uses junction tables to handle all many to many relationships, including person_group, project_member,

task_assignment, project_literature, and expt_literature. This avoids redundancy and ensures that relationship specific attributes like role, join_date, and assigned_date are stored on the relationship where they belong rather than duplicated across entity tables. Referential integrity is enforced through foreign keys, and additional constraints such as valid status values, bounded progress, and basic email format checks help maintain consistent data.

The access control model is also a strength. Views such as member_contact_view and project_access_view restrict what different users can see, and database roles (rg_reader, rg_member, rg_admin) enforce permissions at the database level instead of relying only on the application. From a design perspective, centering the schema around the project entity works well since it naturally connects tasks, experiments, literature, and group membership. The UI is consistent across all pages, which makes navigation straightforward whether the user is viewing projects, tasks, or members.

Revisions & Improvements

Discuss revisions made to your database application. It should address:

- What changes your group made based on feedback
- Why these changes were made
- How they improved the application

Several improvements were made based on feedback and earlier iterations. One of the main changes was ensuring that relationship specific attributes such as role and join_date are stored in junction tables like person_group and project_member. These attributes depend on the full relationship rather than a single entity, so placing them in junction tables keeps the schema consistent and avoids update anomalies.

Another change was addressing the multi valued authors attribute in the literature table. It was originally stored as a comma separated field, which violated 1NF. This was removed, eliminating redundancy and improving consistency, even though a full author and literature_author structure was not included in the final schema. Additional refinements included adding CHECK constraints across multiple tables. Status fields for project, task, and experiment are restricted to valid values, task.progress is limited to a 0 to 100 range, email follows a basic format, and date ordering ensures end_date comes after start_date. Query accuracy was also improved using COUNT(DISTINCT) to prevent overcounting in joins. These changes made the database more consistent and better aligned with normalization principles.

Team Contributions

Discuss how your group worked together. It should address:

- Each group member's role
- Contributions each group member made

Work was split across the database and application, with each person focusing on different parts of the system. Sofia spearheaded the application development, including building the PHP interface, connecting it to the database, and implementing user creation, permissions, and UI flows, while also managing the repository. Keely focused on database design, including the E-R diagram, normalization, and query development, and contributed to UI features such as view pages and linking interface elements. Logan worked on schema development, generating sample data, and parts of the UI and database setup. Sean contributed to security-related components and supported database development and documentation. Bamba focused on optimization and reliability, including indexing, query performance, and transaction handling. Work was coordinated through the shared repository, with changes across schema, queries, and the application closely tied together.

Lessons Learned

Discuss the challenges you faced and what you learned. It should address:

- Challenges encountered during development
- Lessons learned from building the application

One of the main takeaways from this project was the importance of communication and coordination. Since the database, queries, and application are tightly connected, small schema changes can break other parts of the system if they are not synchronized. Being more consistent about syncing changes earlier would have reduced rework, especially when local SQL files and the deployed database did not match.

Another key lesson was how normalization decisions impact implementation. Choices like where to store attributes such as role or how to structure many to many relationships directly affect query complexity and data consistency. The project also showed the impact of query design, where replacing nested IN queries with joins reduces unnecessary execution steps, which was visible when comparing EXPLAIN output for different query versions.

Future Improvements

Discuss potential future improvements. It should address:

- What you would improve or add if you had more time

With more time, the application could be expanded in both functionality and structure. Adding filtering, search, and sorting across all tables would improve usability beyond basic data display. The schema could be extended with a proper author and literature_author relationship to fully model authorship instead of omitting it.

Performance could be improved by adding indexes on frequently used columns such as project.status and foreign keys used in joins. Some queries in the application could also be rewritten to replace remaining nested IN patterns with joins. On the interface side, better error handling and more dynamic interactions, such as inline editing or confirmation prompts, would make the system easier to use in practice.